Vetri Index



Name Mohammed Dhariq. I. B

Subject CS23532 - Computer Networks

Std ..... Sec ..... Roll No. 241001137

School .....

NO	DATE	TITLE	MARKS	SIGNATURE
1.	17/07/26	Linux and Windows commands		
2.	17/07/26	Learning and assignment of IP address Manually		
3.	23/07/26	Types of Network Cables and crimping of cable with RJ45		
4.	30/7/26	Implement packet sniffing using raw sockets in python		
5.	24/07/26	Develop a customized ping command to test the server connectivity		
9.	13/08/26	Design a simple topology and configure with one router, two switches and PCs		
10.a	14/08/26	Internetworking with routers in cisco packet tracer simulator		
10.b	14/08/26	Internetwork using wireless router, DHCP server and internet cloud.		

Exp-no : 01 Date: 7/07/2026	Basic Networking commands in Linux and Windows operating Systems.

Aim:

To study and understand the basic networking commands in Linux and windows operating systems.

Commands observed:

S.no	Command	Linux	Windows
1.	Display IP configuration	ifconfig (or) ip addr	ip.config
2.	check network connectivity	ping <Last>	ping <host>
3.	Trace route	traceroute <host>	tracert <host>
4.	Display routing table	route -n (or) ip route.	route print
5.	Display network connections	net stat - a	netstat - a
6.	DNS lookup	nslookup <domain>	nslookup <domain>
7.	Display ARP cache	arp - a	arp - a

Outputs

1.	IPv4 Address . . . . . : 192.168.1.100
2.	Reply from 142.250.183.110: bytes = 32 time=18ms TTL=117
3.	Trace complete. Total hops: 10
4.	0.0.0.0.0.0.0.0 192.168.1.100
5.	TCP 0.0.0.0:80 0.0.0.0:0 LISTENING
6.	Name: example.com Address: 93.184.216.34
7.	192.168.1.1 00-1a-2b-3c-4b-5e dynamic
8.	DESKTOP-7HHK2L9
9.	TCP 127.0.0.1:49735 0.0.0.0:0 LISTENING 1024
10.	Server: dns.google Address: 8.8.8.8
11.	Default server: dns.google
12.	Default Server: dns.google
13.	TCP 127.0.0.1:49735 0.0.0.0:0 LISTENING 1024
14.	<!doctype html><html>...
15.	100 1256 100 1256 0 0 1256 0 --:--:--:--:--:-- 1256
16.	PktMon successfully started.
17.	Computing statistics for 100 seconds...
18.	The following commands are available: adv firewall bridge dhcp dnsclient interface wlan...
19.	00-15-5D-01-02-03 \Device\Tcpip_{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}
20.	State: connected.

8.	Display hostname	hostname	hostname.
9.	ss - tuln	ss - tuln	net stat - ano
10.	dig DNS wokup	dig <domain>	ns look up<domain>
11.	DNS information	dig	ns lookup
12.	Host lookup	host	n slookup
13.	Socket Statistics	ss	net stat - ano
14.	Download web content	curl	curl
15.	Download Files	wget	curl - o
16.	Packet capture	tcp dump	pktmon
17.	network diagnostic	mtr	pathping
18.	network management	nmcli	netsh
19.	network adapter info	eth tool	getmac
20.	wireless configuration	iwconfig	netsh wlan show interfaces.

Result:

The basic networking commands in linux and windows were successfully studied and observed.

Exp.no: 2 Date: 17/07/26	Learning and Assignment of IP Address Manually to computers

Aim:

To learn how to manually assign an IP address, subnet mask, default gateway, and DNS server to a computer and verify the network configuration.

Procedure:

1. Open control panel → Network and Sharing center
2. Select change adapter settings.
3. Right click the active network adapter and select properties.
4. Open Internet Protocol version 4 (TCP/IPv4)
5. Select Use the following IP address.
6. Enter the following values:
   - IP Address: 192.168.1.10
   - Subnet Mask: 255.255.255.0

Output:

```
C:\> ipconfig

IPv4 Address . . . . . : 192.168.1.10
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
DNS Servers . . . . . : 8.8.8.8
```

- Default Gateway : 192.168.1.1
- Preferred DNS Server : 8.8.8.8

7. Click ok and save the settings.

8. Open command prompt and execute the ipconfig command.

9. Verify that the manually assigned IP address, subnet mask, default gateway, and DNS server are displayed correctly.

Result:

The IP address was successfully assigned manually, and the system displayed the configured IP address, subnet mask, default gateway and DNS server correctly. The network config was verified using the ~~ip~~ ipconfig command.

Exp no: 3 Date: 23/07/26	Study of different types of Network cables and crimping of cable with RJ45 connector

Aim:

To study different types of network cables and perform crimping of a UTP cable using an RJ45 connector.

Materials Required:

- UTP (Cat5e / Cat6) Cable
- RJ45 connectors
- Crimping tool
- Wire stripper
- cable cutter
- LAN cable tester

Types of Network Cables:

- UTP (Unshielded Twisted Pair): Most commonly used for Ethernet networks.
- STP (Shielded Twisted pair): Has shielding to reduce electromagnetic interference.
- Coaxial Cable: Used for cable TV and older networking systems.

TYPES OF NETWORK CABLES

Choose the Right Cable for Reliable & High-Performance Networks

1 UNSHIELDED TWISTED PAIR (UTP) CABLES



Most common cable for LAN, Ethernet, and office networks.

STRUCTURE



Outer Jacket



Twisted Pairs

KEY BENEFITS

- Commonly used in homes and offices
- Flexible, affordable, and easy to use
- For Ethernet and VoIP applications

2 SHIELDED TWISTED PAIR (STP) CABLES



Ideal for environments with high EMI and interference.

STRUCTURE



Outer Jacket



Foil Shielding



Twisted Pairs

KEY BENEFITS

- Protect against interference and crosstalk
- Require proper grounding
- Ideal for environments with heavy EMI such as factories and industrial setups

3 COAXIAL CABLES



Used for traditional broadband, CCTV, and RF transmissions.

STRUCTURE



Outer Protective Jacket



Metallic Shield



Copper Conductor

KEY BENEFITS

- Highly resistant to signal obstruction
- Great for long-distance transmissions
- Commonly used in data centers, ISP networks, and radio/broadcast applications

4 FIBER OPTIC CABLES



Built for speed and performance in modern networks.

STRUCTURE



Outer Jacket



Glass Fiber Strands

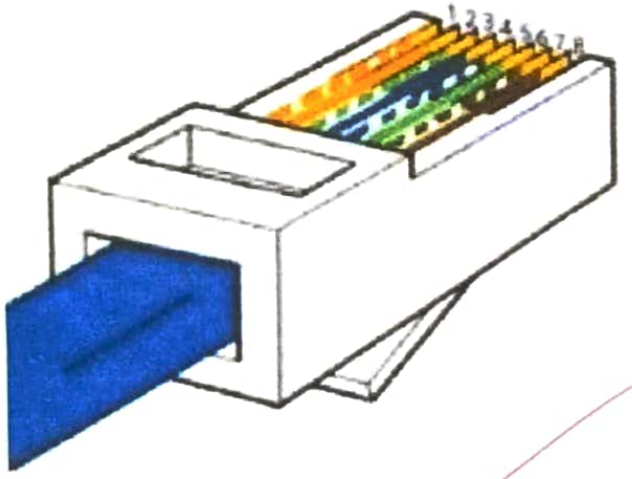


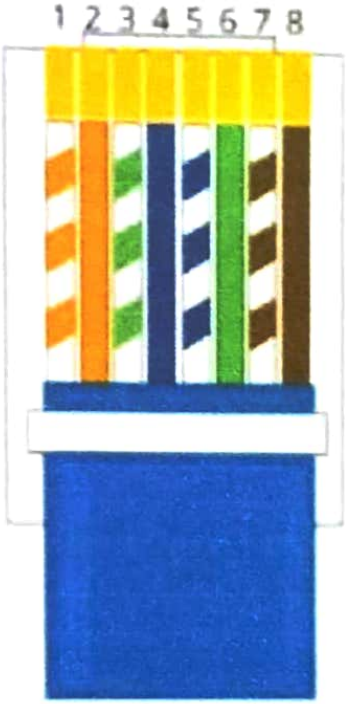
Light Signal Transmission

KEY BENEFITS

- Immune to electromagnetic interference
- Ideal for ultra-fast, long-range data transmission
- Widely used in data centers, ISP networks, and high-bandwidth applications

Tip: Selecting the right cable improves network performance, reduces downtime, and ensures a reliable network infrastructure.





1. White Orange	5. White Blue
2. Orange	6. Green
3. White Green	7. White Brown
4. Blue	8. Brown

- Fiber optic cable: uses light to transmit data at very high speeds over long distances.

Crimping Procedure:-

* Cut the cable to the required length
* Strip about 2-3 cm of the outer insulation
* Untwist and arrange the wires according to the T568B standard:
  - White/Orange range
  - Orange
  - White/Green
  - Blue
  - White/Blue
  - Green
  - White/Brown
  - Brown
* Trim the wires evenly
* Insert the wires fully into the RJ45 connector
* Use the crimping tool to crimp the connector
* Test the cable using a LAN cable tester.

Result:

The network cable was successfully crimped with an RJ45 connector and tested for proper connectivity.

Ex.no:4 Date:30/7/26	Implement Packet Sniffing using Raw sockets in python

Aim:

To implement a packet sniffer using raw sockets in python that captures and analyzes network packets by extracting basic IP header information.

Requirements:-

- Python 3.x
- Linux (or) Windows
- root/ Administrator

A packet sniffer is a program that captures packets travelling across a network interface. It uses raw sockets.

- [x] creates a raw socket
- [x] capturing incoming packets
- [x] Extracts the IP header
- [x] Displays information such as source IP, destination IP, protocol, and packet length.

Procedure:

* Open a terminal or command prompt with Root/ Admin privileges.
* Install Python 3 if it is not already installed
* Create a python file named packet sniffer.py.
* Copy and paste following code
* Save the file
* Run the program
* Generate some network traffic (use the ping command)
* Observe the captured packet details.

Code

```
import socket
import struct

s = socket.socket (socket.AF_INET, socket.SOCK_RAW,
                   socket.IPPROTO_IP)

host = socket.gethostbyname (socket.gethostname())

s.bind ((host, 0))

s.setsockopt (socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
```

Output:

Packet Sniffer started...

Source IP : 192.168.1.10

Destination IP : 142.250.183.206

Version : 4

Header Length : 20 bytes

TTL : 64

Protocol : 6

Packet Length : 60 bytes

Source IP : 192.168.1.10

Destination IP : 8.8.8.8

Version : 4

Header length : 20 bytes

TTL : 128

Protocol : 17

Packet Length : 72 bytes

```
try:
    s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

except:
    pass

print("Packet Sniffer started...\n")

while True:
    packet, addr = s.recvfrom(65535)
    ip_header = packet[:20]
    iph = struct.unpack('!BBHHHBBH4s4s', ip_header)

    version = iph[0] >> 4
    ihl = iph[0] & 0xF
    ttl = iph[5]
    protocol = iph[6]
    src_ip = socket.inet_ntoa(iph[8])
    dst_ip = socket.inet_ntoa(iph[9])
    print("Source IP:", src_ip)
    print("Destination IP:", dst_ip)
    print("version:", version)
    print("Header Length:" ihl * 4, "bybes")
    print("TTL:", ttl)
    print("Protocol:", protocol)
    print("Packet Length:", len(packet), "bybes")
    print("-" * 50)
```

Result:

Thus Packet sniffing was implemented using raw sockets. Network packet details were captured successfully.

Ex.no: 5 Date: 24/07/26	Develop a customized ping command to test the server connectivity

Aim:

To develop and execute a customized ping command to test the connectivity between the client system and the target server, and to analyze the response time and packet loss.

1) Why are we using the ping command?

The ping command is used to verify whether a destination server or host is reachable over a network.

2) Purpose of the ping command?

* Test network connectivity between two devices.
* Check if a server is online
* Measure Response time and detect packet loss.

3) How is it established?

The source sends an ICMP Echo request to the destination. If the destination is reachable, it returns an ICMP echo reply, confirming network connectivity.

Output:

Pinging google.com [142.251.222.174] with 32 bytes of data:

Reply from 142.251.222.174: bytes = 32 time=9ms TTL=119

Reply from 142.251.222.174: bytes = 32 time=9ms TTL=119

Reply from 142.251.222.174: bytes = 32 time=7ms TTL=119

Reply from 142.251.222.174: bytes = 32 time=20ms TTL=119

Ping statistics for 142.251.222.174:

Packets: Sent = 4 Received = 4 Lost = 0 (0% loss),

Approximate round trip times in milli-seconds:

Minimum = 7ms, Maximum = 20ms, Average = 11ms

Procedure:

1. Open the command prompt (windows) or Terminal (macOS/Linux)
2. Enter the customized ping command with packet size

   windows:

   ping google.com

   ping google.com -l 1024

   Linux/macOS

   ping -c 5 google.com
3. Press Enter to execute the command
4. Observe the ICMP Echo request and Echo reply messages.
5. Note the response time (RTT), packet loss and TTL values
6. Analyze the results to determine whether the server is reachable and the network connection is stable.

Result:

The customized ping command was successfully executed. The server responded to all requests with 0% packet loss, confirming that the server was reachable and the connection is stable.

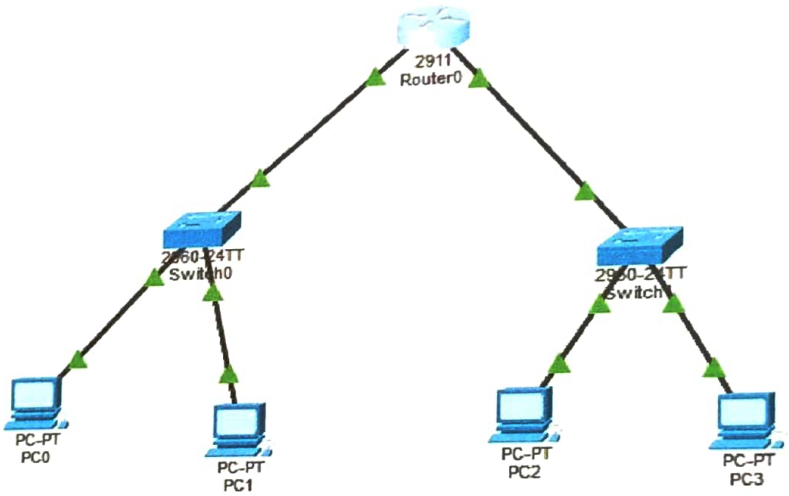
Ex.no: 9	Implementation of Subnetting in
Date: 13/08/26	cisco packet tracer

Aim:

To implement classless IP subnetting in Cisco Packet Tracer and verify communication between different subnets.

Procedure:

1. Open cisco packet tracer and create a new network topology.
2. Add the required devices such as routers, switches, and PCs to the workspace.
3. Connect the devices using appropriate cables. Connect PCs to switches and switches to router interfaces.
4. Use the network 192.168.1.0/24 and divide it into smaller subnets using a /27 subnet mask (255.255.255.224).
5. A /27 subnet provides 30 usable host addresses. The required networks are:


2911
Router0
2960-24TT
Switch0
2960-24TT
Switch1
PC-PT
PC0
PC-PT
PC1
PC-PT
PC2
PC-PT
PC3

(i) 192.168.1.0/27

(ii) 192.168.2.0/27

(iii) 192.168.3.0/27

(iv) 192.168.4.0/27

6. Configure the router interface through the CLI. Enter;

```
enable
configure terminal
interface FastEthernet 0/0
ip address 192.168.1.1 255.255.255.224
no shutdown
exit

interface FastEthernet 0/1
ip address 192.168.2.1 255.255.255.224
no shutdown
exit
```

7. Configure the switch ports in access mode:

```
enable
configure terminal
interface FastEthernet 0/1
switchport mode access
exit
interface FastEthernet 0/2
switchport mode access
exit.
```

```
Router>enable
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.2.1     YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Reply from 192.168.1.10: bytes=32 time=3ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time=6ms TTL=128
Reply from 192.168.1.10: bytes=32 time=4ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 6ms, Average = 3ms

C:\>
```

* Configure each PC with the correct IP address, subnet mask, and default gateway.

* Configure all router interfaces according to the topology

* Check router interface status and ensure they are up and using ping from each PC to test connectivity with its gateway.

* Successful ping confirms the subnetting and network configuration are correct.

Observation:

a) understanding of subnetting: It divides a large network into smaller networks. In this experiment 127(255.255.255.224) gives 32 addresses and 30 hosts.

b) Advantages of Subnetting:

* Efficient IP usage, Reduces network traffic, Improve performance, improve security and Easier troubleshooting.

c) Subnetting in college network:

It separates departments, labs etc.

Ex: 192.168.1.0/27, 192.168.2.0/27

Result:

Thus, the 192.168.100/24 network was successfully divided into 127 subnets, configured in packet tracer and connectivity was verified using ping.

Ex.no: 10.a Date: 14/08/26	Internetworking with router in CISCO PACKET TRACER simulator

Aim:

To design and configure a simple internetwork using a router and two PCs in Cisco packet tracer and verify connectivity using a simple PDU

Procedure:

Step 1: Configure Router1

- Open Router1 → CLI and enter:

```
Router > enable
Router # config t
Router (config) # interface FastEthernet 0/0
Router (config-if) # ip address 192.168.10.1 255.255.255.0
Router (config-if) # no shutdown
Router (config-if) # interface FastEthernet 0/1
Router (config-if) # ip address 192.168.20.1 255.255.255.0
Router (config-if) # no shutdown
```

Step 2: Configure PCs

- Go to PC → Desktop → IP configuration
- Configure the following:

Device	IP Address	Subnet Mask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1


Switch-PT-Empty
SW0
PC-PT
PC0

Step 3: Connect Devices:

- connect PC0 FastEthernet 0 → Router1 Fa0/0
- connect PC1 FastEthernet 0 → Router 1 Fa0/1
- Use copper straight-through cables.
- Send a simple PDU from PC0 and PC1 to verify connectivity.

Router configuration:

Derice	Fa0/0 IP	Subnet Mask	Fa0/1 P	Subnet Mask
Router 1	192.168.10.1	255.255.255.0	192.168.20.1	255.255.255.0

Result:

The internetwork was successfully designed and configured. The simple PDU was successfully transferred from PC0 and PC1, confirming successful connectivity between the two networks.

Ex.No:10.b	Internetwork using wireless router, DHCP
Date: 14/08/26	Server and internet cloud.

Aim:

To design and configure an internetwork using a wireless Router, DHCP Server and Internet Cloud in cisco packet Tracer and verify connectivity.

Procedure:

Step 1: Build the Network

- [x] Place PC, Wireless Router, Laptop, Cable Modem, Internet cloud and cisco.com Server.
- [x] Rename the devices as required
- [x] Connect
  - PC → Wireless Router using copper cable
  - Wireless → Cable modem using copper cable
  - Cable modem → Internet cloud using Coaxial cable.
  - Internet Cloud → cisco.com using copper cable.

Step 2: Configure Wireless router

- Set SSID : Home Network
- Enable DHCP Server
- Set DNS Server : 208.67.220.220
- Save the settings.

Step 3 : Configure Laptop

- Replace Ethernet module with wireless WPC300N
- Power ON the laptop
- Go to Desktop → PC Wireless
- Select and connect to HomeNetwork

Step 4: Configure internet cloud

- Install PT-CLOUD-NM-1CX and PT-CLOUD-NM-1CFE
- configure coaxial → Ethernet connection
- Set the Ethernet provider network as cable

Step 5: Verify connectivity

- On PC, use :

  ipconfig / release

  ip config / renew

- verify the PC receives a 192.168.0.2x address
- Test connectivity using :

  ping cisco.com

- Successful replies confirm connectivity

Observation

1. Key features of wireless Router & DHCP

- Wireless Router: Provides Wi-Fi, SSID, LAN and Internet connectivity.
- DHCP : Automatically provides network settings, reduces . manual configuration and prevents IP

3. Internetwork Design

PC - Wireless Router - cableModem - Internet Cloud - Server

|

Laptop

IP configuration

Device	IP Address	Gateway
PC	DHCP (192.168.0.x)	192.168.0.1
Wireless Router	192.168.0.1	-
Laptop	DHCP	192.168.0.1
cisco.com Server	208.67.220.220	208.67.220.1

Result:

The internetwork was successfully designed and configured using a wireless Router, DHCP server and Internet cloud. The PC and Laptop obtained IP addresses through DHCP, and the ping to cisco.com was successful, confirming network connectivity.

Practical 11

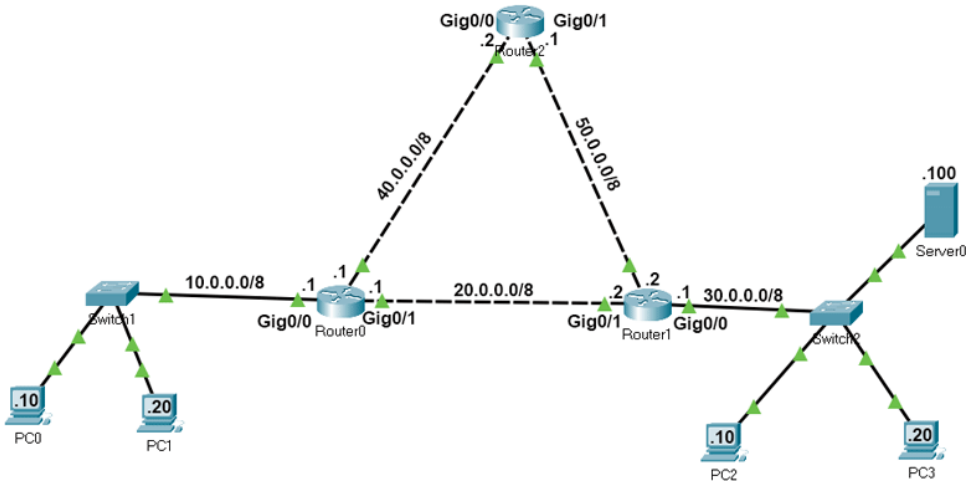
AIM:- a) Simulate Static Routing Configuration using CISCO Packet Tracer

Static routes are the routes you manually add to the router's routing table. The process of adding static routes to the routing table is known as static routing. Let's take a Packet Tracer example to understand how to use static routing to create and add a static route to the routing table.

Setting up a practice lab

Create a Packet Tracer lab as shown in the following image or download the pre-created lab and load it on Packet Tracer.

Packet Tracer Lab with Initial IP Configuration



In this lab, each network has two routes to reach. We will configure one route as the main route and another route as the backup route. If the link bandwidth of all routes is the same, we use the route that has the least number of routers as the main route. If the link bandwidth and the number of routers are the same, we can use any route as the main route and another route as the backup route.

If we specify two routes for the same destination, the router automatically selects the best route for the destination and adds the route to the routing table. If you manually want to select a route that the router should add to the routing table, you have to set the AD value of the route lower than other routes. For example, if you use the following commands to create two static routes for network 30.0.0.0/8, the router will place the first route in the routing table.

```
#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
```

If the first route fails, the router automatically adds the second route to the routing table.

Creating, adding, verifying static routes

Routers automatically learn their connected networks. We only need to add routes for the networks that are not available on the router's interfaces. For example, networks 10.0.0.0/8, 20.0.0.0/8 and 40.0.0.0/8 are directly connected to Router0. Thus, we don't need to configure routes for these networks. Networks 30.0.0.0/8 and 50.0.0.0/8 are not available on Router0. We have to create and add routes only for these networks.

The following table lists the connected networks of each router.

Router	Available networks on local interfaces	Networks available on other routers' interfaces
Router0	10.0.0.0/8, 20.0.0.0/8, 40.0.0.0/8	30.0.0.0/8, 50.0.0.0/8
Router1	20.0.0.0/8, 30.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 40.0.0.0/8
Router2	40.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 20.0.0.0/8, 30.0.0.0/8

Let's create static routes on each router for networks that are not available on the router.

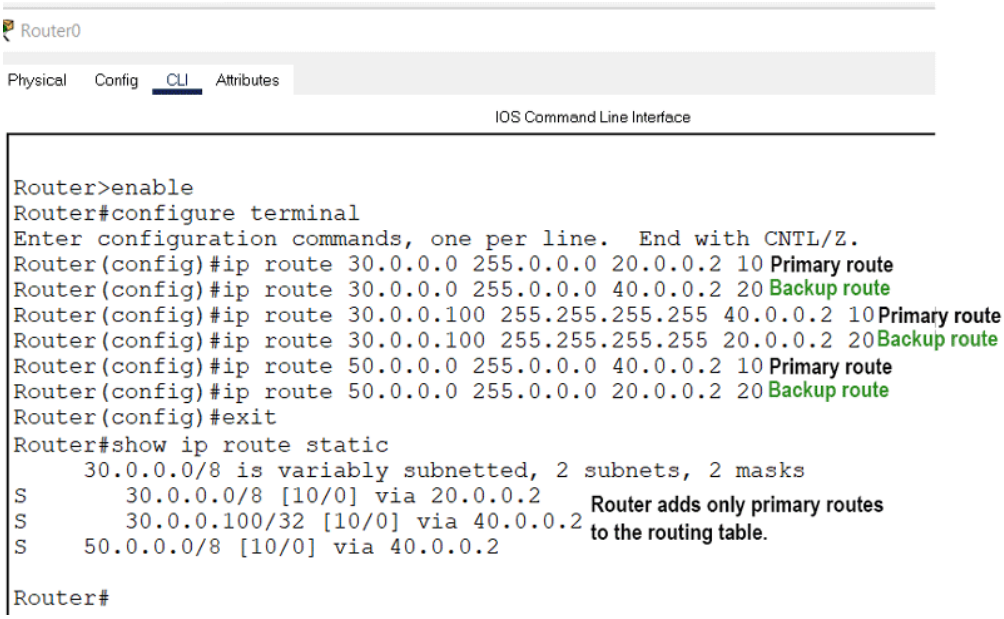
Router0 requirements

- Create two routes for network 30.0.0.0/8 and configure the first route (via Router1) as the main route and the second route (via Router2) as a backup route.
- Create two routes for the host 30.0.0.100/32 and configure the first route (via Router2) as the main route and the second route (via Router1) as a backup route.
- Create two routes for network 50.0.0.0/8 and configure the first route (via Router2) as the main route and the second route (via Router1) as a backup route.
- Verify the router adds only main routes to the routing table.

Router0 configuration

Access the CLI prompt of Router0 and run the following commands.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20
Router(config)#exit
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
Router#
```

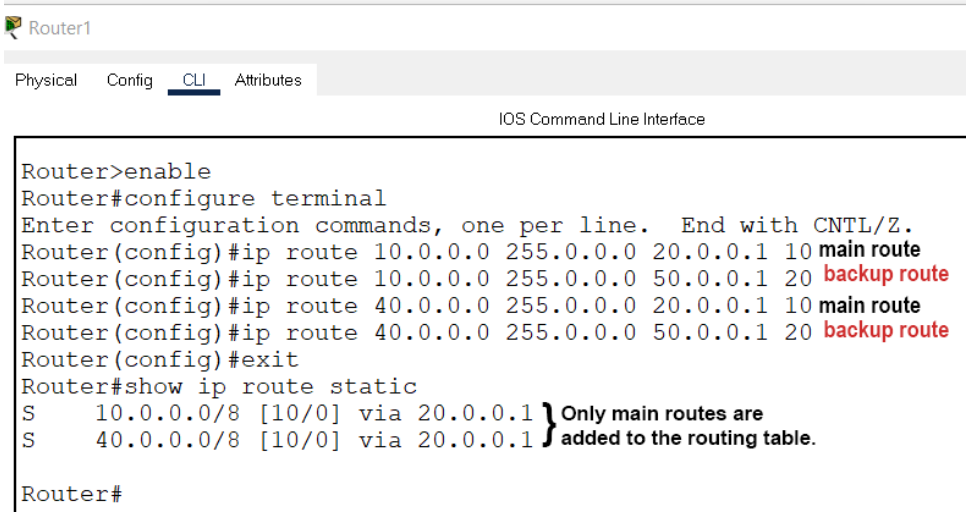


Router1 requirements

- Create two routes for network 10.0.0.0/8 and configure the first route (via Router0) as the main route and the second route (via Router2) as a backup route.
- Create two routes for network 40.0.0.0/8 and configure the first route (via Router0) as the main route and the second route (via Router2) as a backup route.
- Verify the router adds only main routes to the routing table.

Router1 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 10.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#ip route 40.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 40.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [10/0] via 20.0.0.1
S 40.0.0.0/8 [10/0] via 20.0.0.1
Router#
```

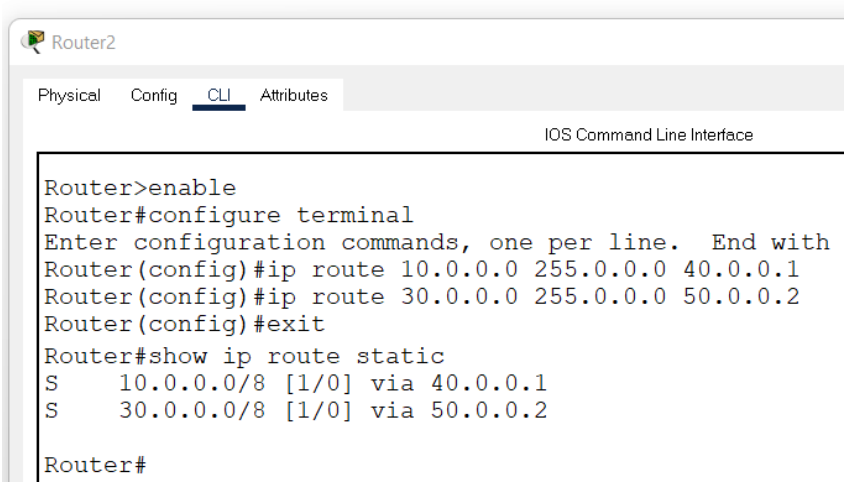


Router2 requirements

Create static routes for network 10.0.0.0/8 and network 30.0.0.0/8 and verify the router adds both routes to the routing table.

Router2 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 40.0.0.1
Router(config)#ip route 30.0.0.0 255.0.0.0 50.0.0.2
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [1/0] via 40.0.0.1
S 30.0.0.0/8 [1/0] via 50.0.0.2
Router#
```



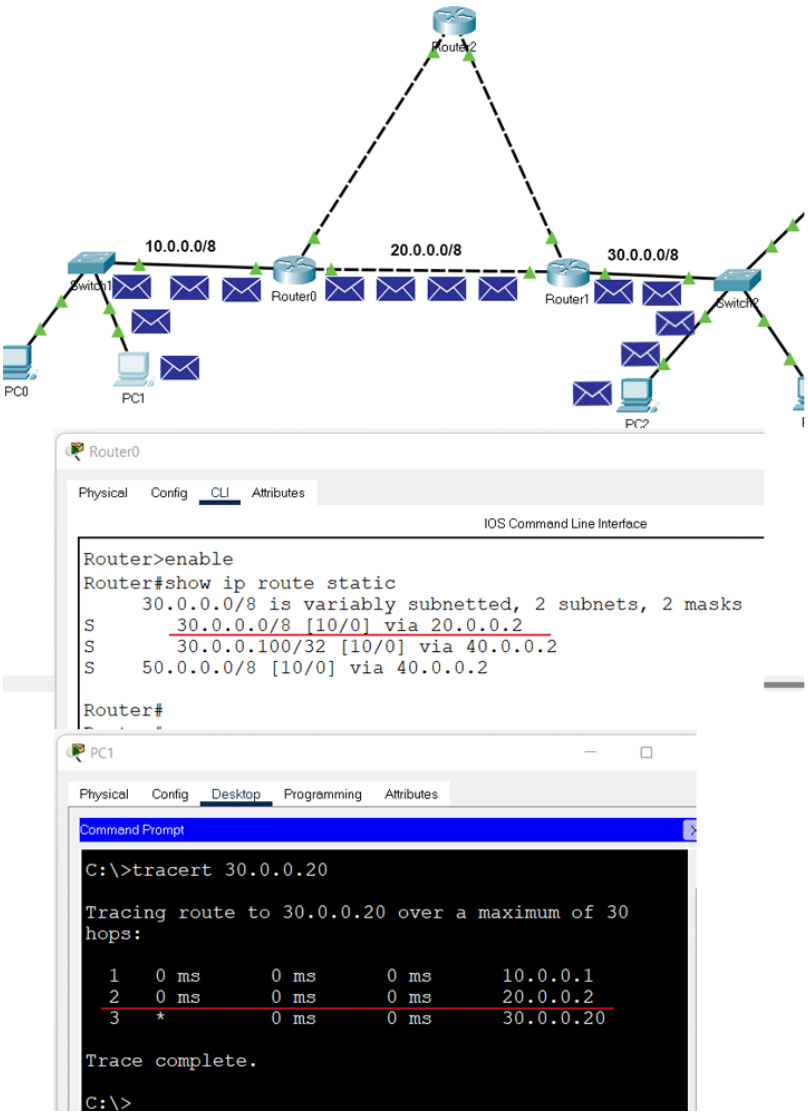
Verifying static routing

On Router0, we configured two routes for network 30.0.0.0/8. These routes are via Router1 and via Router2. We set the first route (via Router1) as the main route and the second route as the backup route. We can verify this configuration in two ways.

By sending ping requests to a PC of network 30.0.0.0/8 and tracing the path they take to reach it. For this, you can use the ‘tracert’ command on a PC of network 10.0.0.0/8. The ‘tracert’ command sends ping requests to the destination host and tracks the path they take to reach the destination.

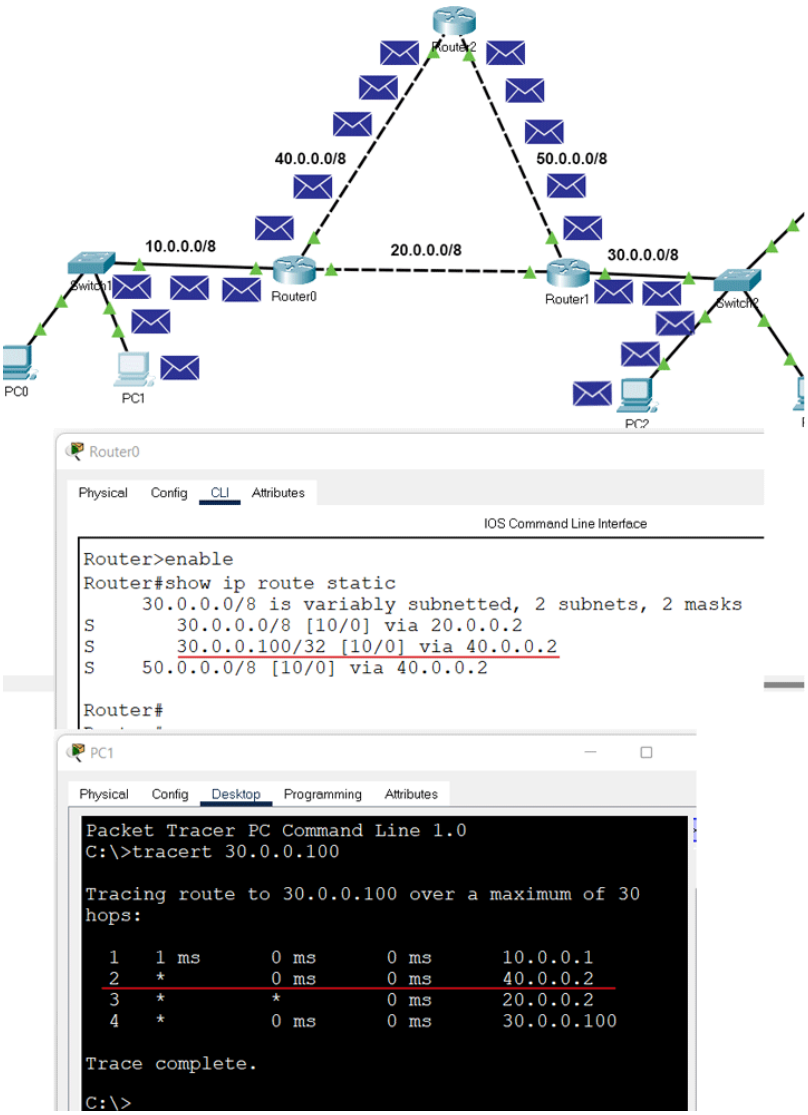
By listing the routing table entries on Router0. Since a router uses the routing table to forward data packets, you can check the routing table to figure out the route the router uses to forward data packets for each destination.

The following image shows the above testing.



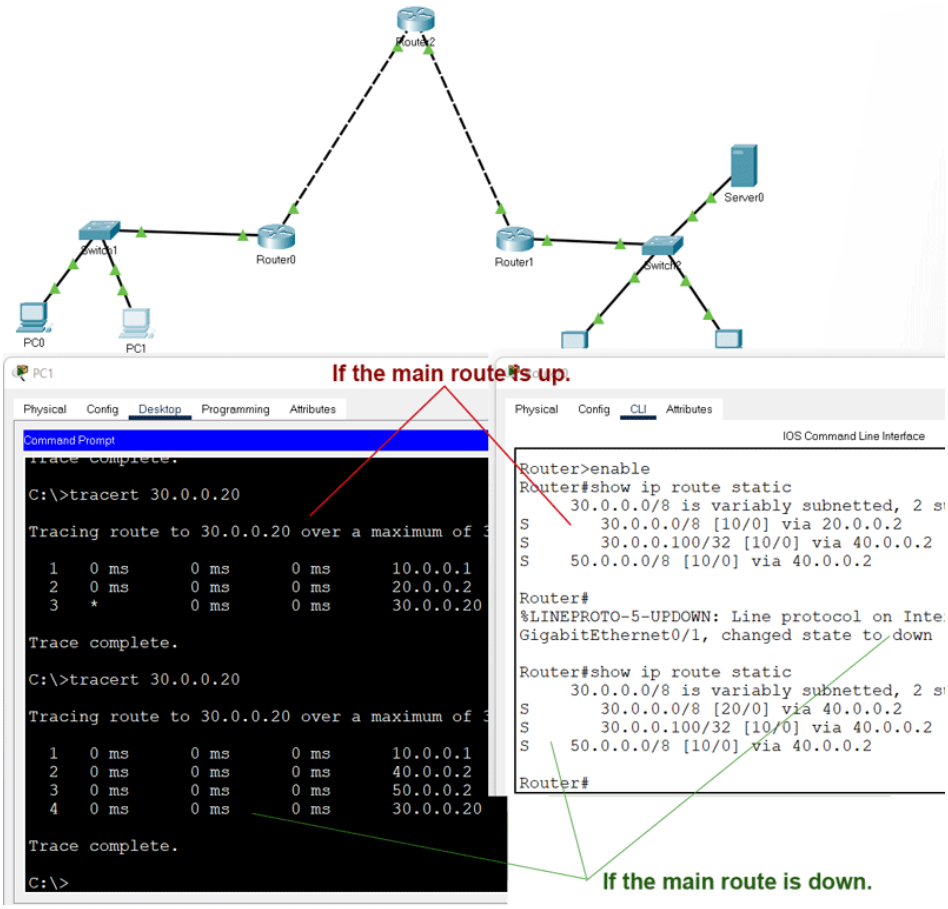
We also configured a separate static host route for the host 30.0.0.100/32. The router must use this route to forward data packets to the host 30.0.0.100. To verify this, you can do the same testing for the host 30.0.0.100.

The following image shows this testing.



We also configured a backup route for network 30.0.0.0/8. The router must put the backup route in the routing table and use it to forward data packets to network 30.0.0.0/8 when the main route fails. To verify this, we have to simulate the failure of the main route.

To simulate the failure of the main route, you can delete the link between Router0 and Router1. After deleting the link, do the same testing again for the network 30.0.0.0/8.



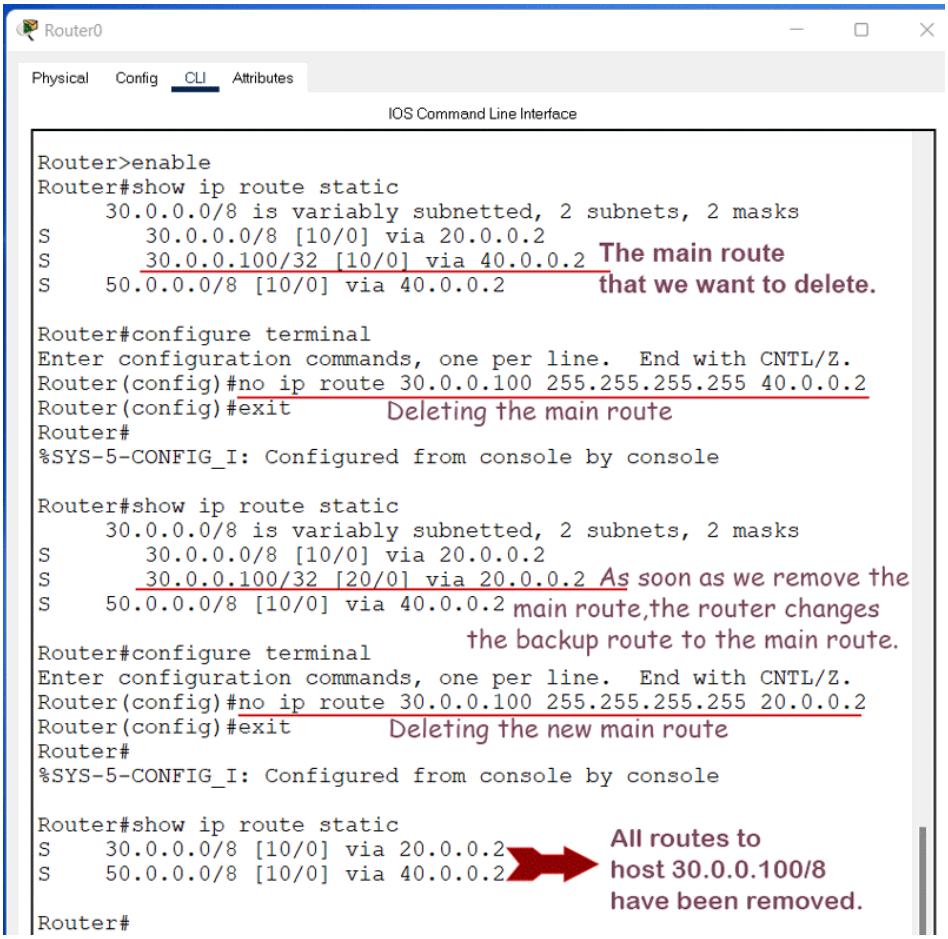
Deleting a static route

To delete a static route, use the following steps.

- Use the ‘show ip route static’ command to print all static routes.
- Note down the route you want to delete.
- Use the ‘no ip route’ command to delete the route.

If you have a backup route, the backup route becomes the main route when you delete the main route.

In our example, we have a backup route and a main route for the host 30.0.0.100/32. The following image shows how to delete both routes.


Router0
Physical
Config
CLI
Attributes
IOS Command Line Interface
Router>enable
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [10/0] via 40.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
The main route that we want to delete.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 40.0.0.2
Router(config)#exit
Deleting the main route
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [20/0] via 20.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
As soon as we remove the main route,the router changes the backup route to the main route.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 20.0.0.2
Router(config)#exit
Deleting the new main route
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip route static
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
All routes to host 30.0.0.100/8 have been removed.
Router#

Practical 11

AIM:- b) Simulate RIP using CISCO Packet Tracer

Initial IP configuration

Device	Interface	IP Configuration	Connected with
PC0	Fast Ethernet	10.0.0.2/8	Router0's Fa0/1
Router0	Fa0/1	10.0.0.1/8	PC0's Fast Ethernet
Router0	S0/0/1	192.168.1.254/30	Router2's S0/0/1
Router0	S0/0/0	192.168.1.249/30	Router1's S0/0/0
Router1	S0/0/0	192.168.1.250/30	Router0's S0/0/0
Router1	S0/0/1	192.168.1.246/30	Router2's S0/0/0
Router2	S0/0/0	192.168.1.245/30	Router1's S0/0/1
Router2	S0/0/1	192.168.1.253/30	Router0's S0/0/1
Router2	Fa0/1	20.0.0.1/30	PC1's Fast Ethernet
PC1	Fast Ethernet	20.0.0.2/30	Router2's Fa0/1

Assign IP address to PCs

Double click the PCs, click the Desktop menu item and click IP Configuration. Assign the IP address referring to the above table.

Assign IP address to interfaces of routers

Double click Router0, click CLI and press the Enter key to access the command prompt of Router0.

We need to configure an IP address and other parameters on interfaces before we can actually use them for routing. Interface mode is used to assign an IP address and other parameters. Interface mode can be accessed from global configuration mode. The following commands are used to access global configuration mode.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#
```

From global configuration mode we can enter interface mode and configure the interface. The following commands assign an IP address on FastEthernet0/0.

```
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 10.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

- interface fastEthernet 0/0 command is used to enter interface mode.
- ip address 10.0.0.1 255.0.0.0 command will assign the IP address to the interface.
- no shutdown command will bring the interface up.
- exit command is used to return to global configuration mode.

A serial interface needs two additional parameters: clock rate and bandwidth. Every serial cable has two ends, DTE and DCE. These parameters are always configured at the DCE end. We can use the show controllers interface command from privileged mode to check the cable's end.

```
Router#show controllers serial 0/0/0
Interface Serial0/0/0
Hardware is PowerQUICC MPC860
DCE V.35, clock rate 2000000
[Output omitted]
```

The third line of output confirms that the DCE end of the serial cable is attached. If you see DTE here instead of DCE, skip these parameters. Now we have the necessary information, so let's assign IP addresses to the serial interfaces of Router0.

```
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.249 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.254 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

- Router#configure terminal – enters global configuration mode.
- Router(config)#interface serial 0/0/0 – enters interface mode.
- Router(config-if)#ip address 192.168.1.249 255.255.255.252 – assigns the IP address to the interface. For a serial link we usually use an IP address from a /30 subnet.
- clock rate 64000 and bandwidth 64 – in a real environment these parameters control the data flow between serial links and are set at the service provider's end. In a lab environment we can simply use these values.
- no shutdown – brings the interface up.
- exit – returns to global configuration mode.

We will use the same commands to assign IP addresses on the interfaces of the remaining routers. We need to provide clock rate and bandwidth only on the DCE side of a serial interface.

Router1

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.250 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.246 255.255.255.252
Router(config-if)#clock rate 64000
```

```
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
```

Use the same commands to assign IP addresses on the interfaces of Router2.

Router2

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 20.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.245 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.253 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
```

Now the routers have information about the networks that they have on their own interfaces. Routers will not exchange this information between themselves on their own. We need to implement the RIP routing protocol that will make them share this information.

Configure RIP routing protocol

Configuration of the RIP protocol requires only two steps.

- Enable the RIP routing protocol from global configuration mode.
- Tell the RIP routing protocol which networks you want to advertise.

Let's configure it on Router0.

Router0

```
Router0(config)#router rip
Router0(config-router)# network 10.0.0.0
Router0(config-router)# network 192.168.1.252
Router0(config-router)# network 192.168.1.248
```

The router rip command tells the router to enable the RIP routing protocol. The network command allows us to specify the networks which we want to advertise. We only need to specify the networks which are directly connected to the router. That is all we need to configure RIP. Follow the same steps on the remaining routers.

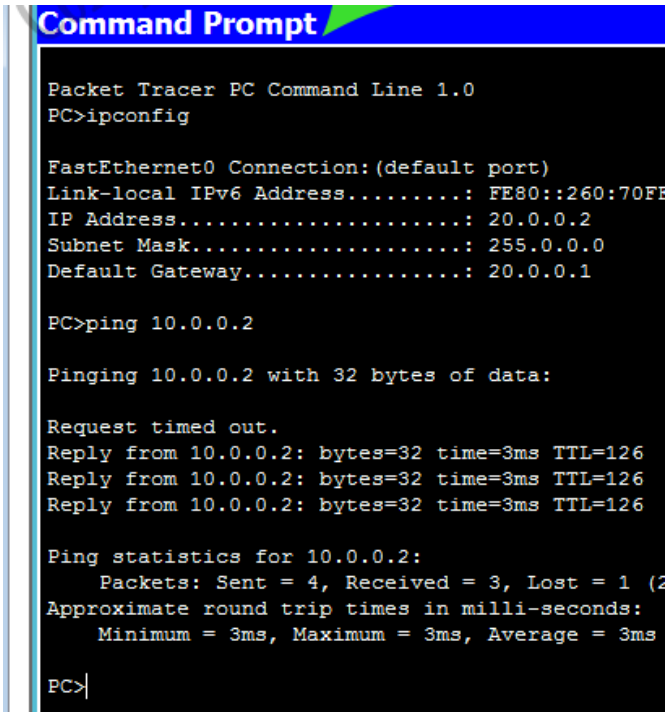
Router1

```
Router1(config)#router rip
Router1(config-router)# network 192.168.1.244
Router1(config-router)# network 192.168.1.248
```

Router2

```
Router2(config)#router rip
Router2(config-router)# network 20.0.0.0
Router2(config-router)# network 192.168.1.252
Router2(config-router)# network 192.168.1.244
```

That's it. Our network is ready to take advantage of RIP routing. To verify the setup we will use the ping command, which tests the connectivity between two devices. Access the command prompt of PC1 and use the ping command to test the connectivity to PC0.



RIP automatically manages all routes for us. If one route goes down, it automatically switches to another available route. To explain this process more clearly we have added one more route in our network. Currently there are two routes between PC0 and PC1.

Route 1

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/1 – 192.168.1.254] <==> Router2 [Serial0/0/1 – 192.168.1.253] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination / source – 20.0.0.2]

Route 2

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/0 – 192.168.1.249] <==> Router1 [Serial0/0/0 – 192.168.1.250] <==> Router1 [Serial0/0/1 – 192.168.1.246] <==> Router2 [Serial0/0/0 – 192.168.1.245] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination / source – 20.0.0.2]

By default RIP will use the route that has the lowest hop count between source and destination. In our network Route 1 has the lower hop count, so it will be selected. We can use the tracert command to verify it.

Now suppose Route 1 is down. We can simulate this situation by removing the cable attached between Router0 [S0/0/1] and Router2 [S0/0/1]. RIP will automatically reroute the traffic. Use the tracert command again to see dynamic routing in action.

Practical 12

AIM:- a) Implement echo client server using TCP/UDP sockets.

Algorithm:-

Server

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Bind the socket to the server IP address and a port number (e.g., 127.0.0.1:8000).
3. Call listen() to wait for incoming client connections.
4. Call accept() to accept a connection from a client.
5. Receive the message from the client using recv().
6. Send the same message back to the client using send() (echo).
7. Repeat steps 5–6 until the client closes the connection or sends “exit”.
8. Close the client socket and the server socket.

Client

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Connect to the server using the server IP address and port number.
3. Read a message from the user.
4. Send the message to the server using send().
5. Receive the echoed message from the server using recv() and display it.
6. Repeat steps 3–5 until the user types “exit”.
7. Close the socket.

Program (Python):-

echo_server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 8000))
s.listen(1)
print("Server is waiting for a connection...")
conn, addr = s.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data or data.decode().lower() == "exit":
        break
    print("Received:", data.decode())
    conn.send(data)          # echo back
conn.close()
s.close()
```

echo_client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 8000))
while True:
    msg = input("Enter message (exit to quit): ")
```

```
    c.send(msg.encode())
    if msg.lower() == "exit":
        break
    data = c.recv(1024)
    print("Echo from server:", data.decode())
c.close()
```

Input:-

Messages typed at the client terminal:

```
Hello Server
Computer Networks Lab
exit
```

Output:-

Server terminal

```
Server is waiting for a connection...
Connected by ('127.0.0.1', 52814)
Received: Hello Server
Received: Computer Networks Lab
```

Client terminal

```
Enter message (exit to quit): Hello Server
Echo from server: Hello Server
Enter message (exit to quit): Computer Networks Lab
Echo from server: Computer Networks Lab
Enter message (exit to quit): exit
```

Practical 12

AIM:- b) Implement chat client server using TCP/UDP sockets.

Algorithm:-

Server

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Bind the socket to the server IP address and port number.
3. Call listen() and then accept() to establish a connection with the client.
4. Receive a message from the client using recv() and display it.
5. Read a reply from the server user and send it to the client using send().
6. Repeat steps 4–5 until either side sends “bye”.
7. Close the connection and the server socket.

Client

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM).
2. Connect to the server using the server IP address and port number.
3. Read a message from the client user and send it to the server.
4. Receive the server’s reply using recv() and display it.
5. Repeat steps 3–4 until either side sends “bye”.
6. Close the socket.

Program (Python):-

chat_server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 9000))
s.listen(1)
print("Chat server waiting for client...")
conn, addr = s.accept()
print("Client connected:", addr)
while True:
    msg = conn.recv(1024).decode()
    if not msg or msg.lower() == "bye":
        print("Client left the chat.")
        break
    print("Client:", msg)
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "bye":
        break
conn.close()
s.close()
```

chat_client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 9000))
while True:
```

```
    msg = input("Client: ")
    c.send(msg.encode())
    if msg.lower() == "bye":
        break
    reply = c.recv(1024).decode()
    if not reply or reply.lower() == "bye":
        print("Server left the chat.")
        break
    print("Server:", reply)
c.close()
```

Input:-

Client

```
Hi
How are you?
bye
```

Server

```
Hello
I am fine
```

Output:-

Server terminal

```
Chat server waiting for client...
Client connected: ('127.0.0.1', 53102)
Client: Hi
Server: Hello
Client: How are you?
Server: I am fine
Client left the chat.
```

Client terminal

```
Client: Hi
Server: Hello
Client: How are you?
Server: I am fine
Client: bye
```

Practical 13

AIM:- Implement your own ping program

Algorithm:

1. Create a RAW socket using socket(AF_INET, SOCK_RAW, IPPROTO_ICMP) (requires administrator / root privileges).
2. Resolve the destination host name to an IP address.
3. Build an ICMP Echo Request packet: Type = 8, Code = 0, Identifier, Sequence Number and payload data.
4. Compute the Internet checksum of the packet and insert it into the ICMP header.
5. Record the send time and send the packet to the destination using sendto().
6. Wait for the reply using recvfrom() with a timeout.
7. Extract the ICMP header from the received packet (after the 20-byte IP header); if Type = 0 (Echo Reply) and the identifier matches, calculate the round-trip time (RTT) and TTL.
8. If no reply arrives before the timeout, display “Request timed out”.
9. Repeat for the required number of packets (e.g., 4) and display statistics: packets sent, received, lost, and minimum / maximum / average RTT.

Program (Python):-

```
import socket, struct, time, sys

def checksum(data):
    if len(data) % 2:
        data += b"\x00"
    s = sum(struct.unpack("!%dH" % (len(data) // 2), data))
    s = (s >> 16) + (s & 0xFFFF)
    s += s >> 16
    return ~s & 0xFFFF

def ping(host, count=4, timeout=2):
    ip = socket.gethostbyname(host)
    print(f"Pinging {host} [{ip}] with 32 bytes of data:")
    sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_ICMP)
    sock.settimeout(timeout)
    pid = 1234 & 0xFFFF
    rtts = []
    for seq in range(1, count + 1):
        header = struct.pack("!BBHHH", 8, 0, 0, pid, seq)
        payload = b"a" * 32
        chk = checksum(header + payload)
        header = struct.pack("!BBHHH", 8, 0, chk, pid, seq)
        start = time.time()
        sock.sendto(header + payload, (ip, 0))
        try:
            data, addr = sock.recvfrom(1024)
            rtt = (time.time() - start) * 1000
            ttl = data[8]
            icmp_type, _, _, rid, _ = struct.unpack("!BBHHH", data[20:28])
            if icmp_type == 0 and rid == pid:
                rtts.append(rtt)
```

```
                print(f"Reply from {addr[0]}: bytes=32 time={rtt:.0f}ms
TTL={ttl}")
        except socket.timeout:
            print("Request timed out.")
        time.sleep(1)
    sock.close()
    lost = count - len(rtts)
    print(f"\nPing statistics for {ip}:")
    print(f"    Packets: Sent = {count}, Received = {len(rtts)}, Lost =
{lost} ({lost*100//count}% loss)")
    if rtts:
        print(f"    Minimum = {min(rtts):.0f}ms, Maximum =
{max(rtts):.0f}ms, Average = {sum(rtts)/len(rtts):.0f}ms")

ping(sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1")
```

Input:-

```
sudo python3 myping.py 8.8.8.8
```

Output:-

```
Pinging 8.8.8.8 [8.8.8.8] with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=23ms TTL=117
Reply from 8.8.8.8: bytes=32 time=22ms TTL=117
Reply from 8.8.8.8: bytes=32 time=24ms TTL=117
Reply from 8.8.8.8: bytes=32 time=22ms TTL=117

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
    Minimum = 22ms, Maximum = 24ms, Average = 23ms
```

Practical 14

AIM:- Write a code using RAW sockets to implement packet sniffing.

Algorithm:

1. Create a RAW socket using socket(AF_PACKET, SOCK_RAW, ntohs(0x0003)) so that all Ethernet frames are captured (requires root privileges on Linux).
2. Enter a loop and receive packets using recvfrom(65535).
3. Unpack the first 14 bytes as the Ethernet header: destination MAC, source MAC and EtherType.
4. If EtherType = 0x0800 (IPv4), unpack the next 20 bytes as the IP header and extract version, header length, TTL, protocol, source IP and destination IP.
5. Based on the protocol field: 6 = TCP (extract source / destination ports, sequence number and flags), 17 = UDP (extract ports and length), 1 = ICMP (extract type and code).
6. Display the extracted details for each packet.
7. Continue until the user stops the program (Ctrl + C), then close the socket.

Program (Python):-

```
import socket, struct

def mac(b):
    return ":".join("%02x" % x for x in b)

sniffer = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.ntohs(0x0003))
count = 0
try:
    while True:
        raw, _ = sniffer.recvfrom(65535)
        dst, src, proto = struct.unpack("!6s6sH", raw[:14])
        if proto != 0x0800:          # only IPv4
            continue
        count += 1
        ihl = (raw[14] & 0x0F) * 4
        ttl, p, _, s_ip, d_ip = struct.unpack("!8xBBH4s4s", raw[14:34])
        print(f"\nPacket {count}")
        print(f"Ethernet : {mac(src)} -> {mac(dst)}")
        print(f"IPv4     : {socket.inet_ntoa(s_ip)} ->
{socket.inet_ntoa(d_ip)}  TTL={ttl}")
        data = raw[14 + ihl:]
        if p == 6:
            sp, dp = struct.unpack("!HH", data[:4])
            print(f"Protocol : TCP   Src Port={sp}  Dst Port={dp}")
        elif p == 17:
            sp, dp = struct.unpack("!HH", data[:4])
            print(f"Protocol : UDP   Src Port={sp}  Dst Port={dp}")
        elif p == 1:
            t, c = struct.unpack("!BB", data[:2])
            print(f"Protocol : ICMP  Type={t}  Code={c}")
        else:
            print(f"Protocol : {p}")
except KeyboardInterrupt:
    print("\nSniffing stopped. Packets captured:", count)
    sniffer.close()
```

Input:-

Run the sniffer with root privileges and generate some traffic from another terminal (e.g., ping or opening a web page):

```
sudo python3 sniffer.py
ping 8.8.8.8
```

Output:-

```
Packet 1
Ethernet : 3c:52:82:aa:10:5f -> a4:5e:60:c1:22:7b
IPv4     : 192.168.1.10 -> 8.8.8.8  TTL=64
Protocol : ICMP  Type=8  Code=0

Packet 2
Ethernet : a4:5e:60:c1:22:7b -> 3c:52:82:aa:10:5f
IPv4     : 8.8.8.8 -> 192.168.1.10  TTL=117
Protocol : ICMP  Type=0  Code=0

Packet 3
Ethernet : 3c:52:82:aa:10:5f -> a4:5e:60:c1:22:7b
IPv4     : 192.168.1.10 -> 142.250.183.14  TTL=64
Protocol : TCP   Src Port=51522  Dst Port=443

^C
Sniffing stopped. Packets captured: 3
```

Practical 15

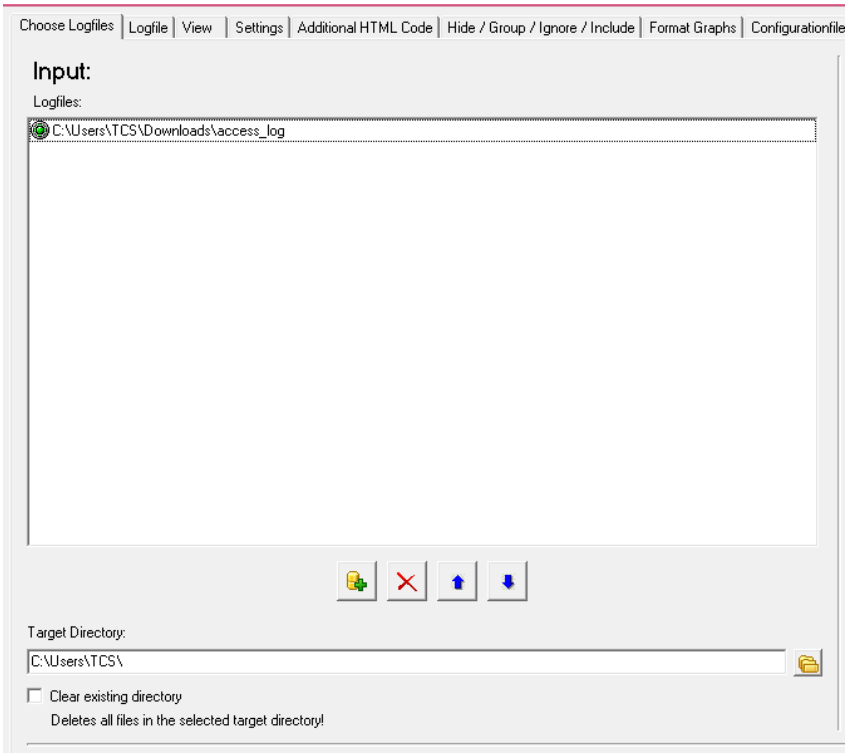
AIM:- To analyze the different types of web logs using Webalizer tool.

Procedure

Step 1: Run the Webalizer Windows version.

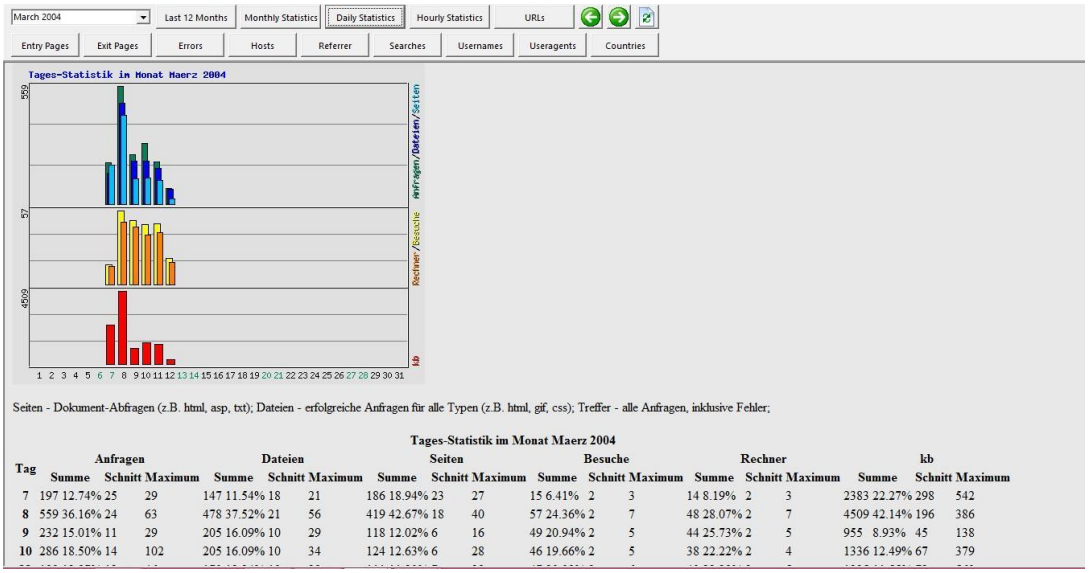
Step 2: Input the web log file (download from the web).

Step 3: Press Run Webalizer.



Output:

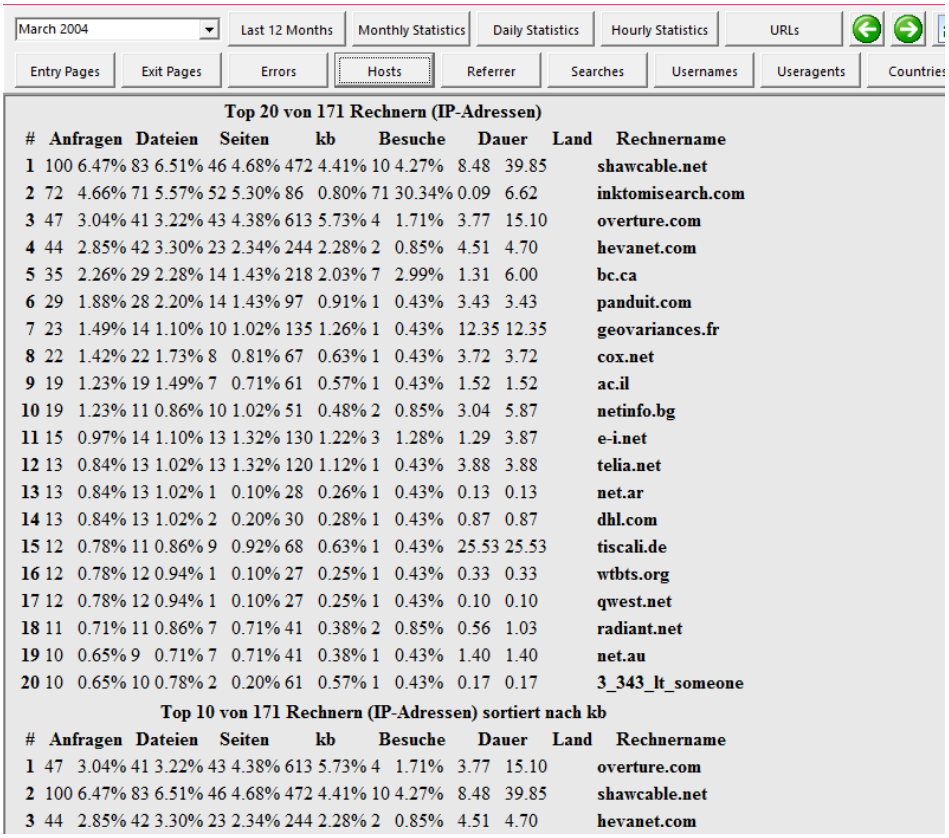
Monthly statistics



Tages-Statistik im Monat Maerz 2004

Tag	Anfragen			Dateien			Seiten			Besuche			Rechner			kb		
	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum	Summe		Schnitt	Maximum
7	197	12.74%	25	29	147	11.54%	18	21	186	18.94%	23	27	15	6.41%	2	3	14	8.19%	2	3	2383	22.27%	298	542
8	559	36.16%	24	63	478	37.52%	21	56	419	42.67%	18	40	57	24.36%	2	7	48	28.07%	2	7	4509	42.14%	196	386
9	232	15.01%	11	29	205	16.09%	10	29	118	12.02%	6	16	49	20.94%	2	5	44	25.73%	2	5	955	8.93%	45	138
10	286	18.50%	14	102	205	16.09%	10	34	124	12.63%	6	28	46	19.66%	2	5	38	22.22%	2	4	1336	12.49%	67	379

Hosts



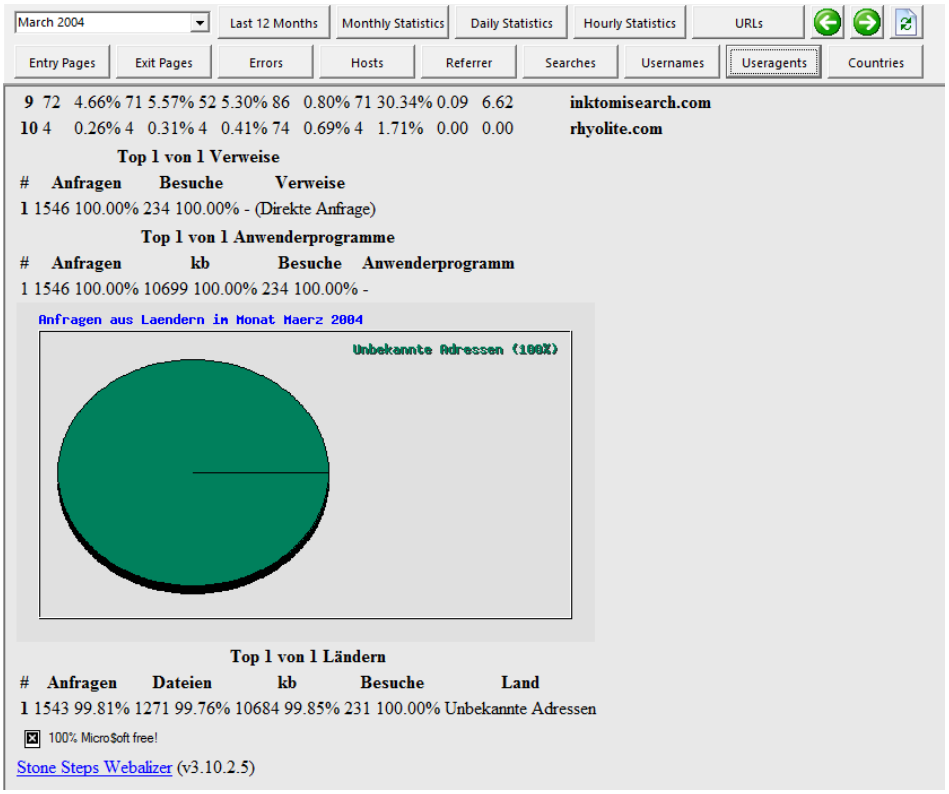
Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer		Land	Rechnername
1	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
2	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62		inktomisearch.com
3	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
4	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com
5	35	2.26%	29	2.28%	14	1.43%	218	2.03%	7	2.99%	1.31	6.00		bc.ca
6	29	1.88%	28	2.20%	14	1.43%	97	0.91%	1	0.43%	3.43	3.43		panduit.com
7	23	1.49%	14	1.10%	10	1.02%	135	1.26%	1	0.43%	12.35	12.35		geovariances.fr
8	22	1.42%	22	1.73%	8	0.81%	67	0.63%	1	0.43%	3.72	3.72		cox.net
9	19	1.23%	19	1.49%	7	0.71%	61	0.57%	1	0.43%	1.52	1.52		ac.il
10	19	1.23%	11	0.86%	10	1.02%	51	0.48%	2	0.85%	3.04	5.87		netinfo.bg
11	15	0.97%	14	1.10%	13	1.32%	130	1.22%	3	1.28%	1.29	3.87		e-i.net
12	13	0.84%	13	1.02%	13	1.32%	120	1.12%	1	0.43%	3.88	3.88		telia.net
13	13	0.84%	13	1.02%	1	0.10%	28	0.26%	1	0.43%	0.13	0.13		net.ar
14	13	0.84%	13	1.02%	2	0.20%	30	0.28%	1	0.43%	0.87	0.87		dhl.com
15	12	0.78%	11	0.86%	9	0.92%	68	0.63%	1	0.43%	25.53	25.53		tiscali.de
16	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.33	0.33		wtbts.org
17	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.10	0.10		qwest.net
18	11	0.71%	11	0.86%	7	0.71%	41	0.38%	2	0.85%	0.56	1.03		radiant.net
19	10	0.65%	9	0.71%	7	0.71%	41	0.38%	1	0.43%	1.40	1.40		net.au
20	10	0.65%	10	0.78%	2	0.20%	61	0.57%	1	0.43%	0.17	0.17		3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer		Land	Rechnername
1	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
2	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
3	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com

User-agents



9	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62	inktomisearch.com
10	4	0.26%	4	0.31%	4	0.41%	74	0.69%	4	1.71%	0.00	0.00	rhyolite.com

Top 1 von 1 Verweise

#	Anfragen		Besuche		Verweise
1	1546	100.00%	234	100.00%	- (Direkte Anfrage)

Top 1 von 1 Anwenderprogramme

#	Anfragen		kb		Besuche		Anwenderprogramm
1	1546	100.00%	10699	100.00%	234	100.00%	-

Top 1 von 1 Ländern

#	Anfragen		Dateien		kb		Besuche		Land
1	1543	99.81%	1271	99.76%	10684	99.85%	231	100.00%	Unbekannte Adressen

100% MicroSoft free!

Stone Steps Webalizer (v3.10.2.5)